

TECNOLOGICO NACIONAL DE MÉXICO INSTITUTO TECNOLÓGICO DE TIJUANA

SUBDIRECCIÓN ACADÉMICA DEPARTAMENTO DE SISTEMAS Y COMPUTACIÓN

SEMESTRE:

Enero - Junio 2026

CARRERA:

Ingeniería en Sistemas Computacionales

MATERIA:

Patrones de Diseño

TÍTULO ACTIVIDAD:

Actividad de valor

UNIDAD A EVALUAR:

Unidad 2

NOMBRE Y NÚMERO DE CONTROL DEL ALUMNO:

Ortega Jimenez Jordi Joel 21212012

NOMBRE DEL MAESTRO (A):

Maribel Guerrero Luis

Patrón de diseño Abstract factory

Estructura del patrón en el proyecto

Fábrica abstracta

La clase **FabricaMenuAbstracta** define qué métodos debe tener cualquier fábrica de menús.

```
import { PlatoPrincipal } from '../interfaces/plato-principal.interface';

import { Bebida } from '../interfaces/bebida.interface';

import { Postre } from '../interfaces/postre.interface';

export abstract class FabricaMenuAbstracta {

    abstract crearPlatoPrincipal(): PlatoPrincipal;

    abstract crearBebida(): Bebida;

    abstract crearPostre(): Postre;

}
```

Esta clase no crea objetos concretos, solo define el contrato que deben cumplir las fábricas.

Productos abstractos

Los productos abstractos son las interfaces que representan los elementos del menú.

plato-principal.interface.ts

```
export interface PlatoPrincipal {

    nombre: string;

    descripcion: string;

    precio: number;

}
```

bebida.interface.ts

```
export interface Bebida {  
  nombre: string;  
  tamano: string;  
  precio: number;  
}
```

postre.interface.ts

```
export interface Postre {  
  nombre: string;  
  nivelDulzor: string;  
  precio: number;  
}
```

Estas interfaces garantizan que todos los restaurantes creen productos con la misma estructura.

Fábricas concretas

Cada restaurante implementa la fábrica abstracta y genera su propio menú.

```
import { FabricaMenuAbstracta } from './fabrica-menu.abstracta';  
import { PlatoPrincipal } from '../interfaces/plato-principal.interface';  
import { Bebida } from '../interfaces/bebida.interface';  
import { Postre } from '../interfaces/postre.interface';  
  
export class FabricaMenuMexicano extends FabricaMenuAbstracta {  
  crearPlatoPrincipal(): PlatoPrincipal {  
    return {
```

```
    nombre: 'Tacos al pastor',  
    descripcion: 'Tacos con carne al pastor, piña, cebolla y cilantro',  
    precio: 120,  
};  
}
```

```
crearBebida(): Bebida {  
    return {  
        nombre: 'Agua de jamaica',  
        tamano: '500ml',  
        precio: 35,  
    };  
}
```

```
crearPostre(): Postre {  
    return {  
        nombre: 'Flan',  
        nivelDulzor: 'Medio',  
        precio: 45,  
    };  
}  
}
```

Cada una crea su propia familia de productos.

Cliente del patrón

En tu proyecto, el cliente del patrón es MenuService, porque es la clase que utiliza las fábricas sin depender directamente de los productos concretos.

```
import { Injectable, NotFoundException } from '@nestjs/common';
import { FabricaMenuAbstracta } from '../fabricas/fabrica-menu.abstracta';
import { FabricaMenuMexicano } from '../fabricas/fabrica-menu-mexicano';
import { FabricaMenuJapones } from '../fabricas/fabrica-menu-japones';
import { FabricaMenuItaliano } from '../fabricas/fabrica-menu-italiano';
import { FabricaMenuAmericano } from '../fabricas/fabrica-menu-americano';
import { FabricaMenuChino } from '../fabricas/fabrica-menu-chino';
import { Menu } from '../interfaces/menu.interface';
```

```
@Injectable()
```

```
export class MenuService {

  obtenerMenuPorTipo(tipoRestaurante: string): Menu {

    const fabrica = this.resolverFabrica(tipoRestaurante);

    return {

      restaurante: tipoRestaurante,

      platoPrincipal: fabrica.crearPlatoPrincipal(),

      bebida: fabrica.crearBebida(),

      postre: fabrica.crearPostre(),

    };

  }

}
```

```
private resolverFabrica(tipoRestaurante: string): FabricaMenuAbstracta {  
    switch (tipoRestaurante.toLowerCase()) {  
        case 'mexicano':  
            return new FabricaMenuMexicano();  
        case 'japones':  
            return new FabricaMenuJapones();  
        case 'italiano':  
            return new FabricaMenuItaliano();  
        case 'americano':  
            return new FabricaMenuAmericano();  
        case 'chino':  
            return new FabricaMenuChino();  
        default:  
            throw new NotFoundException(  
                `No existe una fábrica para el restaurante ${tipoRestaurante}`,  
            );  
    }  
}  
}
```

Aquí se observa claramente el patrón:

- MenuService no crea directamente tacos, ramen o chow mein.
- MenuService primero resuelve qué fábrica usar.
- después llama a:
 - crearPlatoPrincipal()
 - crearBebida()
 - crearPostre()

Así obtiene un menú completo sin acoplarse a clases concretas.

Cuando el frontend solicita un menú, por ejemplo:

GET /menu/mexicano

el sistema hace lo siguiente:

1. MenuController recibe la petición.
2. MenuService identifica el tipo de restaurante.
3. MenuService usa resolverFabrica() para seleccionar la fábrica correcta.
4. La fábrica concreta crea:
 - un plato principal
 - una bebida

- un postre

5. El sistema regresa el menú completo al frontend.

Por ejemplo, si el restaurante es mexicano, el resultado será:

```
{  
  "restaurante": "mexicano",  
  "platoPrincipal": {  
    "nombre": "Tacos al pastor",  
    "descripcion": "Tacos con carne al pastor, piña, cebolla y cilantro",  
    "precio": 120  
  },  
  "bebida": {  
    "nombre": "Agua de jamaica",  
    "tamano": "500ml",  
    "precio": 35  
  },  
  "postre": {  
    "nombre": "Flan",  
    "nivelDulzor": "Medio",  
    "precio": 45  
  }  
}
```


Cómo evita usar tanto new

Sin este patrón, el código tendría que crear los objetos manualmente en muchos lugares del sistema, por ejemplo:

```
const plato = new Tacos();
```

```
const bebida = new AguaDeJamaica();
```

```
const postre = new Flan();
```

Y si se quisiera cambiar de restaurante, habría que modificar el código del cliente directamente.

Con Abstract Factory, el cliente solo cambia la fábrica:

```
const fabrica = new FabricaMenuMexicano();
```

y después usa:

```
fabrica.crearPlatoPrincipal();
```

```
fabrica.crearBebida();
```

```
fabrica.crearPostre();
```

Conclusión

El patrón permite generar menús completos de distintos restaurantes de forma organizada y escalable. La clase `FabricaMenuAbstracta` define la estructura común, mientras que las fábricas concretas producen familias de objetos relacionadas según el tipo de cocina. El `MenuService` actúa como cliente del patrón, utilizando la fábrica adecuada para construir el menú sin depender de clases concretas. Gracias a este enfoque, el sistema evita el uso excesivo de `new`, mejora la mantenibilidad del código y facilita la incorporación de nuevas cocinas en el futuro.